

Laboratory Activity:

Runtime Documentations Programming Languages

Oct 9 2026

1. Objective

The activity asks for a program written in several programming languages, with the runtime of each one documented. The languages must cover different type-system capabilities: static and dynamic typing, and strong and weak typing. This document describes the program, how each language was compiled or run, the output produced, and what the results show about each type system.

2. Languages Chosen

One language was chosen for each combination of static/dynamic and strong/weak typing, so that every capability is represented. The four languages also use three different execution strategies, which makes the runtime comparison meaningful.

Language	Static / Dynamic	Strong / Weak	Execution strategy
C	Static	Weak	Ahead-of-time (AOT) compiled to native code
Rust	Static	Strong	Ahead-of-time (AOT) compiled to native code
Python	Dynamic	Strong	Compiled to bytecode, run on a virtual machine (CPython)
JavaScript	Dynamic	Weak	Just-in-time (JIT) compiled by V8 in Node.js

3. The Program

The same four tests were written in every language. Because the logic is identical, any difference in behavior comes from the language's type system and runtime.

#	Test	What it reveals
1	Add an integer and a string (a char in C)	Whether mixed types are rejected or silently converted
2	Reassign a variable to a different type	Whether a variable's type is fixed (static) or can change (dynamic)
3	Call a function with the wrong argument type	When, or whether, the mismatch is detected
4	Mix an integer and a float	Whether numeric conversion is implicit or must be explicit

All files are in the repository, one folder per language: *c/*, *rust/*, *python/* and *javascript/*.

4. Runtime Documentation

4.1 C (static, weak)

Item	Detail
Version	gcc (Rev4, Built by MSYS2 project) 16.2.0
Compile	gcc main.c -o main
Run	.\main

Type system

- **Static:** every variable has a type fixed at compile time (*int*, *char*), and the compiler checks it before the program runs.
- **Weak:** the compiler converts between types silently. A *char* is treated as a number, and a decimal value is truncated when it is assigned or passed to an *int*.

Execution strategy

Ahead-of-time compilation. gcc translates the source into a native machine-code executable, which runs directly on the CPU. There is no interpreter or virtual machine at run time.

Source code (c/main.c)

```
#include <stdio.h>

int add_one(int n) {
    return n + 1;
}

int main(void) {
    printf("1. int + char\n");
    int x = 5;
    char y = '3';
    printf("    %d\n", x + y);

    printf("2. reassign to a different type\n");
    int v = 10;
    printf("    %d\n", v);
    v = 'A';
    printf("    %d\n", v);

    printf("3. function with wrong type\n");
    printf("    %d\n", add_one(4));
    printf("    %d\n", add_one(3.9));

    printf("4. int + float\n");
    int r = 5 + 2.5;
    printf("    %d\n", r);

    return 0;
}
```

Output

```
1. int + char
56
2. reassign to a different type
10
65
3. function with wrong type
5
4
4. int + float
7
```

Observations

The program compiled and ran without any error, but several values were silently changed. $5 + '3'$ printed 56 because the character '3' is stored as the number 51. Assigning 'A' to an int gave 65. Passing 3.9 to `add_one` printed 4, because the decimal part was truncated to 3 before the addition. `int r = 5 + 2.5` stored 7 instead of 7.5. The types are fixed at compile time (static), yet the compiler allows these conversions without stopping the programmer (weak).

4.2 Rust (static, strong)

Item	Detail
Version	rustc 1.98.1 (48a229cea 2026-09-01)
Compile (corrected)	rustc main.rs then ./main
Compile (failing)	rustc error.rs

Type system

- **Static:** types are checked at compile time. A variable inferred as an integer cannot later hold a string.
- **Strong:** there are no implicit conversions, not even between an integer and a float. Every conversion must be written explicitly (*as f64*, *.parse()*).

Execution strategy

Ahead-of-time compilation. rustc type-checks the entire program and then produces a native executable. If any check fails, no executable is produced and the program never runs.

Source code (rust/error.rs, written the way the other languages allow, to show the compiler's rejection)

```
fn add_one(n: i32) -> i32 {
    n + 1
}

fn main() {
    let a = 5 + "3";

    let mut v = 10;
    v = "hello";

    let b = add_one("a");

    let c = 5 + 2.5;
}
```

Output

```
error[E0277]: cannot add `&str` to `{integer}`
--> error.rs:6:15
   |
6  |     let a = 5 + "3";
   |               ^ no implementation for `{integer} + &str`

error[E0308]: mismatched types
--> error.rs:9:9
   |
8  |     let mut v = 10;
   |               -- expected due to this value
9  |     v = "hello";
   |       ^^^^^^^^ expected integer, found `&str`

error[E0308]: mismatched types
--> error.rs:11:21
   |
11 |     let b = add_one("a");
   |                   ----- ^^^ expected `i32`, found `&str`

error[E0277]: cannot add a float to an integer
--> error.rs:13:15
   |
13 |     let c = 5 + 2.5;
   |               ^ no implementation for `{integer} + {float}`

error: aborting due to 4 previous errors
```

Observations

The four tests were first written the way Python or JavaScript would allow them (*error.rs*). The compiler rejected all four and reported them together, so the program never ran. This shows that Rust checks types statically and refuses to mix incompatible types.

Corrected version

main.rs performs the same ideas with explicit conversions, and it compiles and runs.

```
fn add_one(n: i32) -> i32 {
    n + 1
}

fn main() {
    println!("1. int + string");
    let x: i32 = 5;
    let y: i32 = "3".parse().unwrap();
    println!("{}", x + y);

    println!("2. reassign (same type only)");
    let mut v = 10;
    println!("{}", v);
    v = 20;
    println!("{}", v);

    println!("3. function with correct type");
    println!("{}", add_one(4));

    println!("4. int + float (explicit cast)");
    let f: f64 = 2.5;
    println!("{}", x as f64 + f);
}
```

Output

```
1. int + string
8
2. reassign (same type only)
10
20
3. function with correct type
5
4. int + float (explicit cast)
7.5
```

Observations

To make the program compile, each conversion had to be explicit: the string was parsed into a number, the integer was cast with *as f64*, and the variable could only be reassigned to another integer. Compared with the other languages, Rust moves these mistakes from run time to compile time.

4.3 Python (dynamic, strong)

Item	Detail
Version	Python 3.14.7
Run	python main.py

Type system

- **Dynamic:** types belong to values and are checked while the program runs, so a variable can change from an integer to a string.
- **Strong:** unrelated types are not converted silently, so adding an integer and a string raises a *TypeError*.

Execution strategy

Compiled to bytecode, then run on a virtual machine. Python compiles the source to bytecode and the Python virtual machine executes it. There is no separate compile step for the programmer.

Source code (python/main.py)

```
def add_one(n):
    return n + 1

print("1. int + string")
try:
    print(5 + "3")
except TypeError as e:
    print("    TypeError:", e)

print("2. reassign to a different type")
v = 10
print("    ", v, type(v).__name__)
v = "hello"
print("    ", v, type(v).__name__)

print("3. function with wrong type")
print("    ", add_one(4))
try:
    print("    ", add_one("a"))
except TypeError as e:
    print("    TypeError:", e)

print("4. int + float")
print("    ", 5 + 2.5)
```

Output

```
1. int + string
   TypeError: unsupported operand type(s) for +: 'int' and 'str'
2. reassign to a different type
   10 int
   hello str
3. function with wrong type
   5
   TypeError: can only concatenate str (not "int") to str
4. int + float
   7.5
```

Observations

The program ran line by line, so earlier output appeared before an error was raised. Tests 1 and 3 failed with a *TypeError* at run time, which shows strong typing, but Python only noticed because those lines were executed. Reassigning *v* from an integer to a string worked, which shows dynamic typing. The risky lines were wrapped in *try/except* so the rest of the program

4.4 JavaScript (dynamic, weak)

Item	Detail
Version	Node.js v24.19.0
Run	node main.js

Type system

- **Dynamic:** types belong to values, so *v* can hold a number and later a string.
- **Weak:** the engine coerces values silently instead of reporting an error.

Execution strategy

Just-in-time compilation. Node.js runs on the V8 engine, which interprets the code with a bytecode interpreter and compiles frequently used code to machine code while the program runs.

Source code (javascript/main.js)

```
function addOne(n) {  
    return n + 1;  
}  
  
console.log("1. int + string");  
console.log("  ", 5 + "3", typeof (5 + "3"));  
console.log("  ", 5 - "3", typeof (5 - "3"));  
  
console.log("2. reassign to a different type");  
let v = 10;  
console.log("  ", v, typeof v);  
v = "hello";  
console.log("  ", v, typeof v);  
  
console.log("3. function with wrong type");  
console.log("  ", addOne(4));  
console.log("  ", addOne("a"));  
  
console.log("4. int + float");  
console.log("  ", 5 + 2.5);
```

Output

```
1. int + string  
   53 string  
   2 number  
2. reassign to a different type  
   10 number  
   hello string  
3. function with wrong type  
   5  
   a1  
4. int + float  
   7.5
```

Observations

No test produced an error. `5 + "3"` returned the string "53", while `5 - "3"` returned the number 2, so the same operand types behaved differently depending on the operator. `addOne("a")` returned "a1" instead of failing. This is weak typing: bugs appear as wrong output rather than as errors.

5. Comparison of Results

Test	C	Rust	Python	JavaScript
1. int + string/char	56 (char treated as number)	Compile error	TypeError at run time	"53" (coerced to string)
2. Reassign to another type	char converted to int: 65	Compile error	Allowed	Allowed

Test	C	Rust	Python	JavaScript
3. Function, wrong type	3.9 truncated to 3, result 4	Compile error	TypeError at run time	"a1" (coerced)
4. int + float	Truncated to 7 on assignment	Compile error without a cast	7.5	7.5
When are type errors caught?	Mostly never	Compile time	Run time	Never

6. Conclusion

The same four tests produced four different behaviors. Rust rejected them at compile time, Python rejected them at run time, JavaScript accepted them and coerced the values, and C accepted them and silently changed the values.

Static versus dynamic decides when types are checked: before the program runs (C, Rust) or while it runs (Python, JavaScript). **Strong versus weak** decides whether mismatched types are allowed to mix: C and JavaScript convert them silently, while Rust and Python refuse.

Stricter languages such as Rust trade convenience for earlier and safer error detection, while flexible languages such as JavaScript trade safety for convenience. The execution strategy also differs: C and Rust are compiled ahead of time to native code, Python runs bytecode on a virtual machine, and JavaScript is JIT-compiled by V8.

7. Repository

All source files, per-language runtime notes and terminal screenshots are available at:

<https://github.com/Furina-star/pl-type-system-lab>